

Day 4 — Lean Delivery & Value Stream Mapping

Activity packet for participant quads. Today's job: map the **value stream** for delivering the feature — from idea to value-realized — naming each step's lead time, process time, and flow efficiency. Draft DP Section 4.

Where we are in the week

The outcome map exists (Day 1). The backlog is loaded (Day 2). The tracking plan is wired (Day 3). Today we look at **the system that produces the work**, not the work itself. Most teams optimize the wrong step because they've never seen the whole stream.

By 16:00, every quad has DP Section 4 — a value stream map for the feature's delivery, with lead times measured (or honestly estimated) and queues named.

Inputs

- DP Section 1–Section 3
- The full artifact set (PRD / TCD / TMD / SEP)
- Real or estimated cycle-time data from prior team work (if available)

Lean — the foundational claim

Lean's central insight (from manufacturing, applied to software): **most of the time work spends in a system is waiting**, not being worked on. Reduce waiting; reduce total time. Speed is a function of flow, not effort.

A single user story might take **2 hours of actual coding** but spend **3 weeks** in the system — waiting for spec, waiting for review, waiting for QA, waiting for deploy slot, waiting for someone to merge it.

A TPM who optimizes coding time saves minutes. A TPM who reduces the waiting saves weeks.

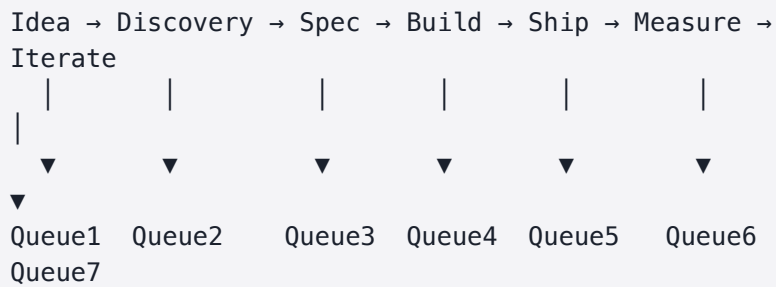
VSM vocabulary card (today's reference)

Term	What it means	Example
Value stream	The whole flow from idea to value-realized	Idea → discovery → spec → build → ship → measure → iterate
Process time (PT)	Hands-on-work time at a step	"2 hours of actual coding"
Lead time (LT)	Total time at a step including waiting	"3 days from PR opened to merged"
Flow efficiency	$PT / LT \times 100\%$	2 hrs / (3 days = 24 hrs) = 8%
Queue	Work waiting between steps	"5 PRs waiting for review"
Cycle time	Lead time across the whole stream	"Idea to production: 6 weeks"
Throughput	Completed items per unit time	"3 stories per sprint"
WIP (Work In Progress)	Items currently being worked	"12 stories started, 4 done"
Little's Law	$\text{Cycle time} = \text{WIP} / \text{throughput}$	More WIP = longer cycle time

The most-startling number is usually **flow efficiency** — single-digit percentages are typical.

A typical software value stream

For a feature like FieldPulse reconcile:



Each step has a **process time** (active work) and a **lead time** (active + queue). The queues are where the time hides.

For most teams, the worst queues are:

- Between **spec** and **build** — waiting for engineering capacity
- Between **build** and **ship** — waiting for code review, QA, deploy slot
- Between **ship** and **measure** — waiting for telemetry to accumulate, for someone to look

Activity 1 — Identify Your Value Stream

Format: Quad • 45 min • Block 1

Purpose

List the steps in the quad's feature delivery stream, before measuring or estimating times.

Quad protocol

1. **Walk the stream from idea to value-realized** (25 min).

For each step:

- What happens here?
- Who does it?
- What's the input? What's the output?
- When does it consider itself "done" so the next step can start?

2. **Identify the queues** (10 min). Between each pair of steps, what's the queue? (Items waiting for the next step to begin.)

3. **Mark the boundaries** (10 min). Where does each step's responsibility end and the next step's begin? (Most teams have ambiguity here — surface it.)

Sample stream — FieldPulse reconcile

Step	What happens	Owner
Idea	Customer signal reaches the team	Customer Success / Sales
Discovery	Quad reviews signal; decides to investigate	TPM
Spec	PRD/TCD/TMD/SEP authored	TPM + quad
Estimate + accept	Engineering accepts into a sprint	Eng Lead
Build	Code + test written	Engineers
Code review	PRs reviewed and merged	Engineers (peer)
QA + integration	Pre-release testing	QA / Engineers
Deploy	Code reaches production	DevOps / on-call
Measure	Outcome metrics + leading indicators reviewed	TPM
Iterate	Decisions about next iteration	TPM + quad

What "good" looks like

- 7–10 steps (more = over-decomposed; fewer = lumping)
- Queues identified between each pair
- Owner per step is **a role** (sometimes a name, sometimes a team)
- The boundary between steps is **explicit**

Activity 2 — Measure or Estimate Each Step

Format: Quad • 50 min • Block 2

Purpose

Add **process time**, **lead time**, and **flow efficiency** numbers to each step.

Quad protocol

For each step:

1. **Process time**: how long does the actual work take, hands-on?
2. **Lead time**: how long does it take *including* the wait until the next step starts?
3. **Flow efficiency**: $PT / LT \times 100\%$

Use real data where available (ADO data; team retros). Otherwise estimate honestly — and label as estimate.

Worked sample

Step	PT	LT	Flow eff	Notes
Idea	1 hour (signal review)	3 days (queue at TPM's calendar)	5%	Signal sometimes lost in inbox
Discovery	4 hours	2 days	25%	Often blocked on customer-interview scheduling
Spec	16 hours	2 weeks	22%	Multiple stakeholder review cycles
Estimate + accept	1 hour	1 week (queue for sprint planning)	3%	One sprint planning per 2 weeks
Build	8 hours per story	1 week per story (mid-sprint)	14%	Multi-tasking; dependency wait
Code review	30 min review	1.5 days waiting for reviewer	4%	Reviewer queue
QA + integration	4 hours	2 days	12%	QA staffing constraint
Deploy	30 min ship	6 hours (deploy windows)	8%	Daily deploy schedule
Measure	1 hour per check	30 days for outcome	n/a	Inherent waiting (data accrual)

Total cycle time idea-to-deploy: ~6–8 weeks. Total flow efficiency for the whole stream: typically 5–15%.

What "good" looks like

- Numbers are **honest**, even if estimated
- Estimates are flagged as estimates

- Flow efficiency is computed (a single-digit number is typical and informative)
 - The team's "feels like" is quantified
-

Activity 3 — Identify the Top 3 Queues

Format: Quad • 50 min • Block 3

Purpose

Find where the time hides. Rank queues by impact.

Quad protocol

1. **Sum the queue times** across the stream (5 min). If you're spending 6 weeks total and only 2–3 days of actual work, where are the other 5+ weeks?
2. **Identify the top 3 queues** (10 min). The longest waits.
3. **Investigate each** (25 min). For each:
 - **Why does this queue exist?** (capacity, policy, inertia, hidden dependency)
 - **What would shrink it?** (more capacity, different policy, parallel work, dependency removal)
 - **Cost of shrinking it?** (people-time, money, organizational change)
4. **Rank the queues** (10 min) by:
 - Impact (how much cycle time would shrink)
 - Effort (how hard to fix)
 - Risk (what could go wrong)

What "good" looks like

- Top 3 queues are specific (not "we're slow in code review" — "PRs wait 2 days for first review because we have 8 reviewers and 40 PRs/week")
- Each has a **named cause** — not just "we're slow"
- Each has a **candidate intervention** — not "be faster"

Worked example (continuing FieldPulse)

Queue	Time	Cause	Candidate intervention
Spec → Estimate (1 week wait)	168 hrs	Sprint planning is bi-weekly	Mid-cycle sync to pre-load stories; OR mid-sprint pull
Build → Code review (1.5 days)	36 hrs	5 reviewers; PRs wait for them	Pair-review during dev; auto-assign reviewer
Discovery → Spec (varies; often weeks)	168+ hrs	Customer interview scheduling	Async interview cycles; lighter-weight discovery for known patterns

Activity 4 — Map the VSM + AI Critique

Format: Quad • 55 min + Wrap • Block 4

Purpose

Draw the visual VSM and use AI to surface non-obvious queues.

The VSM canvas

A typical VSM is drawn as:

[Step 1] → [Step 2] → [Step 3] → ...

PT: __	PT: __	PT: __
LT: __	LT: __	LT: __

QUEUE	QUEUE	QUEUE
__ items	__ items	__ items
__ days	__ days	__ days

Total cycle time: __ days
Total process time: __ days
Total flow efficiency: __%

For a feature delivery, this is one page (whiteboard or paper). The visual makes the queues hit emotionally.

Quad protocol

1. **Draw the VSM** (30 min). On a whiteboard or large paper. Use the canvas above.
2. **Run the AI critique** (10 min):

Role: Lean coach reviewing a software value stream.
Context: <description of the VSM with PT/LT/flow eff numbers and identified queues>
Task: Identify 3 queues likely missing or under-counted:
1. A queue between two steps not yet shown
2. A step where PT and LT seem out of line with team reality
3. An invisible queue (e.g., approvals, "ping someone")
Constraints:
– Use only the data given
– Be specific
Format: 3 numbered findings – what's missing / why suspect / how to verify.

3. **Update the VSM** (10 min). Adopt findings; re-run flow efficiency math.
4. **Document in DP Section 4** (5 min).

Wrap (last 15 min)

Each quad shares:

- Their **flow efficiency number** for the whole stream
- The **single biggest queue** in their stream (by hours)
- One **non-obvious queue** the AI surfaced

End-of-day checkpoint

Each quad ends Day 4 with:

- ✓ Value stream identified (7–10 steps)
- ✓ Process time + lead time + flow efficiency per step
- ✓ **Top 3 queues** named with cause + candidate intervention
- ✓ Visual VSM drawn
- ✓ AI provenance entry

☒ DP Section 4 drafted